

Tutorial 8

Exercise 1: Backtracking for knapsack

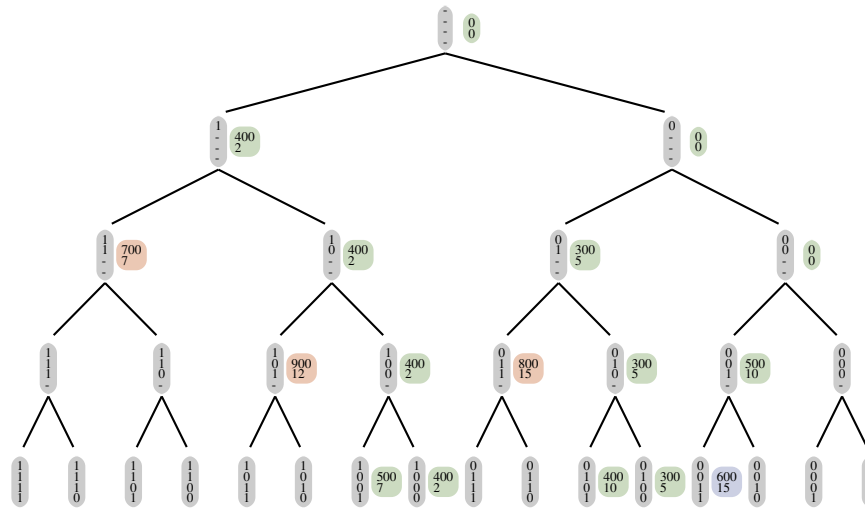
1. Apply the backtracking algorithm for Knapsack to the following instance: $W = 600$, $T = 14$, and

j	1	2	3	4
w_j	400	300	500	100
c_j	2	5	10	5

2. How can the algorithm as presented during the lecture be improved?

Solution:

- 1.



2. Here are four straightforward improvements:

- Items whose weight exceeds the capacity of the knapsack (e.g., item 3 above) can be removed from the list of items, as they will never be part of a valid solution.
- At the last level, the right successor does not have to be explored, as it does not yield a new solution.
- The algorithm as presented during the lecture considers the items in the order they are given in the arrays. So, sorting the items by their average value, i.e., the ratio $\frac{c_j}{w_j}$, ensures that we consider the more promising items first.
- Whenever the value of the remaining items is smaller than $T - \text{cur_v}$, then the remaining subtree does not need to be explored, as no valid solution can be found there.

Exercise 2: The subset sum problem

In the *subset sum problem*, we are given a target $t \in \mathbb{N}$ and a sequence $n_1, n_2, \dots, n_k$ of natural numbers, and the question is whether a subset of the n_j sums up to exactly t . For example, given 1, 2, 3, 4, 5 and $t = 10$ the answer is yes ($1 + 4 + 5 = 10$), given 1, 3, 3, 5, 7 and $t = 2$ the answer is no (note that the 1 in the sequence can only be used once in the sum).

1. Formulate the problem as a decision problem (Hint: recall the definition of Knapsack).
2. Give a backtracking algorithm for the subset sum problem.
3. Apply your algorithm to the instance with $t = 307$ and the sequence 160, 56, 158, 123, 67, 26.

Solution:

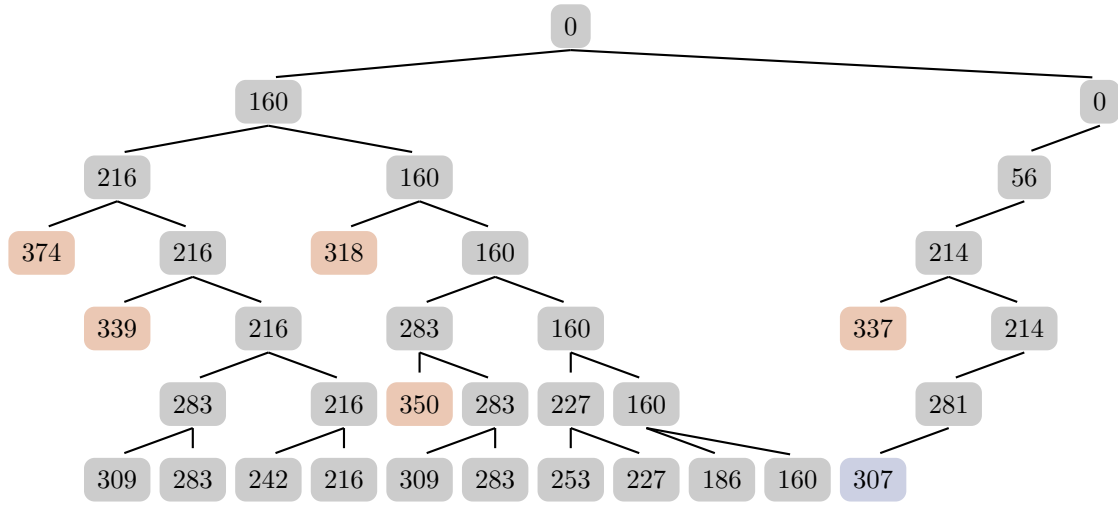
1. $\text{SUBSETSUM} = \{(t, n_1, \dots, n_k) \mid \exists b_1, \dots, b_k \text{ s.t. } \sum_{j=1}^k b_j \cdot n_j = t\}$.
- 2.

```

1 # n is an integer array
2 def subsetsum(t, n):
3     return rec_subsetsum(t, n, 0, 0)
4
5 def rec_subsetsum(t, n, cur_sum, j):
6     # valid solution found
7     if cur_sum == t:
8         return True
9     # target exceeded
10    if cur_sum > t:
11        return False
12    # all items checked
13    if j >= length(n):
14        return False
15    # add j-th number and recursively continue
16    if rec_subsetsum(t, n, cur_sum+n[j], j+1):
17        return True
18    # do not add j-th number and recursively continue
19    if rec_subsetsum(t, n, cur_sum, j+1):
20        return True
21    return False

```

3. We depict the current sum in each node of the search tree, the corresponding partial solutions can be reconstructed from the branching of the tree: a left successor on the j -th level means n_j is added to the sum ($b_j = 1$), a right successor on the j -th level means n_j is not added to the sum ($b_j = 0$).



Exercise 3: DPLL

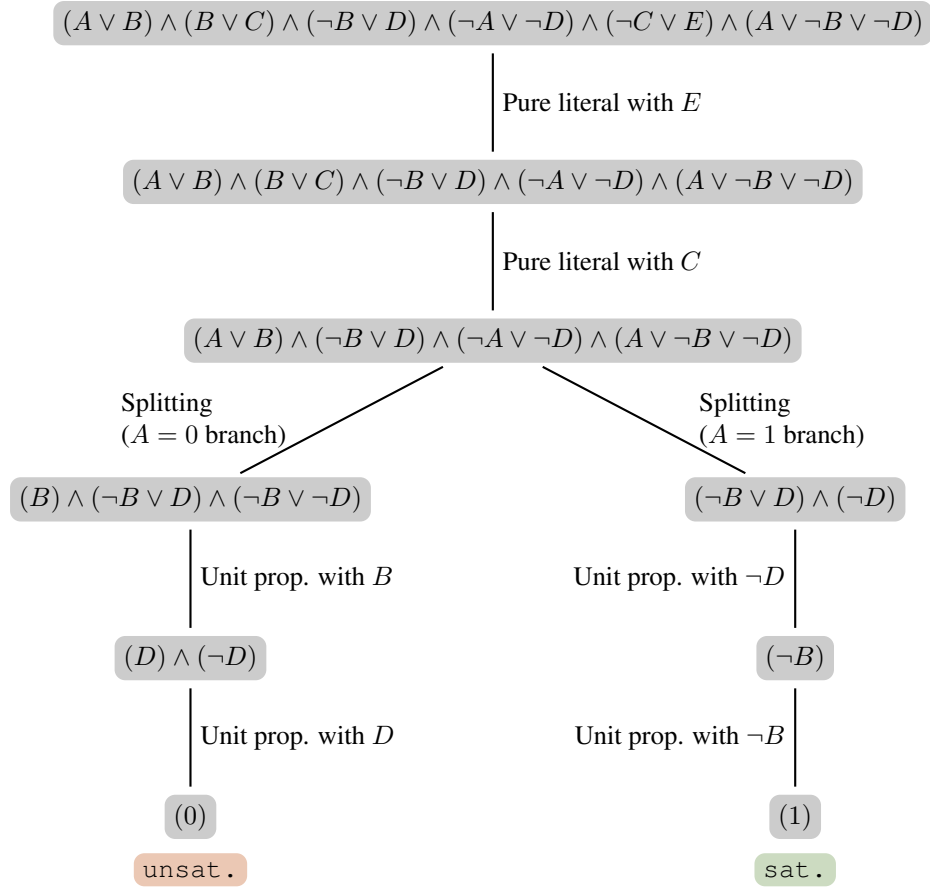
Apply the DPLL algorithm to the following formulas to determine whether they are satisfiable or unsatisfiable.

Illustrate the execution of the algorithm as we did in the lecture using a tree structure, i.e., showing the simplified formula for each recursive call and the rule applied. Assume that DPLL selects variables in alphabetical order (i.e., $A, B, C, D, E, \dots$) if a rule is applicable to more than one variable (e.g., if there is more than one unit clause).

1. $\varphi_1 = (A \vee B) \wedge (B \vee C) \wedge (\neg B \vee D) \wedge (\neg A \vee \neg D) \wedge (\neg C \vee E) \wedge (A \vee \neg B \vee \neg D)$
2. $\varphi_2 = (\neg A \vee \neg B \vee \neg C) \wedge (A \vee \neg B) \wedge (A \vee \neg D) \wedge (B \vee \neg E) \wedge (\neg C \vee D) \wedge (C \vee E) \wedge (C \vee \neg E) \wedge (\neg D \vee E)$

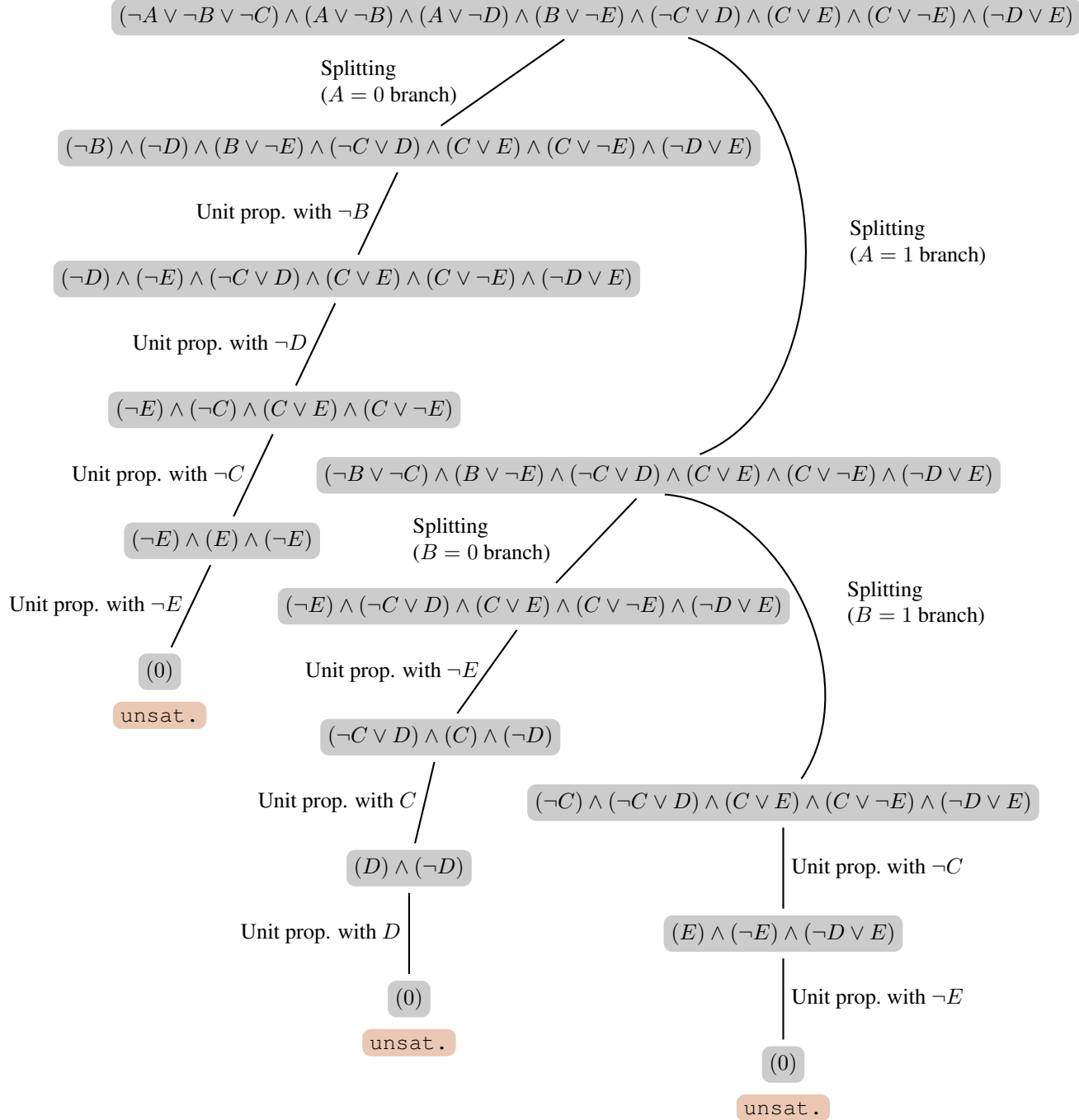
Solution:

1.



So, φ_1 is satisfiable and we can read of a satisfying assignment by collecting the truth values along the path from the satisfiable leaf to the root, e.g., $B = 0$, $D = 0$, $A = 1$, $C = 1$, and $E = 1$.

2.



Exercise 4: Sudoku

Describe how to encode a standard 9×9 Sudoku by an instance of SAT, i.e., given an incomplete Sudoku generate a formula so that a satisfying assignment to the formula encodes a valid completion of the Sudoku.

First, think about which variables to use, then about how to express the required constraints.

Solution:

We use variables of the form s_{xyz} for $1 \leq x, y, z \leq 9$ with the meaning that s_{xyz} is true if and only the entry in row x and column y is the number z .

Now, we express the following constraints:

- There is at least one entry in each cell:

$$\bigwedge_{x=1}^9 \bigwedge_{y=1}^9 \bigvee_{z=1}^9 s_{xyz}$$

- Each number appears at most once in every row:

$$\bigwedge_{y=1}^9 \bigwedge_{z=1}^9 \bigwedge_{x=1}^8 \bigwedge_{i=x+1}^9 (\neg s_{xyz} \vee \neg s_{iyz})$$

Note that we restrict ourselves here to $x < i$ to eliminate some redundancy.

- Each number appears at most once in every column:

$$\bigwedge_{x=1}^9 \bigwedge_{z=1}^9 \bigwedge_{y=1}^8 \bigwedge_{i=y+1}^9 (\neg s_{xyz} \vee \neg s_{xiz})$$

Note that we restrict ourselves here again to $y < i$ to eliminate some redundancy.

- Every number appears at most once in every 3×3 -subgrid:

$$\bigwedge_{z=1}^9 \bigwedge_{i=0}^2 \bigwedge_{j=0}^2 \bigwedge_{x=1}^3 \bigwedge_{y=1}^3 \bigwedge_{k=y+1}^3 (\neg s_{(3i+x)(3j+y)z} \vee \neg s_{(3i+x)(3j+k)z})$$

and

$$\bigwedge_{z=1}^9 \bigwedge_{i=0}^2 \bigwedge_{j=0}^2 \bigwedge_{x=1}^3 \bigwedge_{y=1}^3 \bigwedge_{k=x+1}^3 \bigwedge_{\ell=1}^3 (\neg s_{(3i+x)(3j+y)z} \vee \neg s_{(3i+k)(3j+\ell)z})$$

Here, the first subformula is concerned with cells $y < k$ in the same column of a subgrid while the second one is concerned with cells (x, y) and (k, ℓ) with $x < k$, i.e., all columns to the right. This is again done to avoid redundancy.

- For every clue “in row x_j and column y_j there is number z_j ” there is a unit clause expressing exactly that fact:

$$s_{x_j y_j z_j}$$

A satisfying assignment of the conjunction of these formulas yields a valid sudoku solution and vice versa. This is because if there is at least one number in each cell and no number appears twice in a row, then there is exactly one number in each cell (which is required by the rules, but not explicitly stated in our formula).

However, we can add more constraints that are logically redundant (as they are implied by the above ones), but speed up the solution process.¹

- There is at most one entry in each cell:

$$\bigwedge_{x=1}^9 \bigwedge_{y=1}^9 \bigwedge_{z=1}^8 \bigwedge_{i=z+1}^9 (\neg s_{xyz} \vee \neg s_{xyi})$$

¹See “Inês Lynce and Joël Ouaknine. Sudoku as a SAT Problem.” at <https://people.mpi-sws.org/~joel/publications/sudoku05.pdf> for details.

- Each number appears at least once in each row:

$$\bigwedge_{y=1}^9 \bigwedge_{z=1}^9 \bigvee_{x=1}^9 s_{xyz}$$

- Each number appears at least once in each column:

$$\bigwedge_{x=1}^9 \bigwedge_{z=1}^9 \bigvee_{y=1}^9 s_{xyz}$$

- Each number appears at least once in each 3×3 -subgrid:

$$\bigwedge_{z=1}^9 \bigwedge_{i=0}^2 \bigwedge_{j=0}^2 \bigvee_{x=1}^3 \bigvee_{y=1}^3 s_{(3i+x)(3j+y)z}$$